

NCBI datasets

We have been working in our markdown document named `run_blast_class.md` that has all the steps to create our `chr21_script.sh` and `blast script`. Briefly, what we have done is:

1. Create our working environments by assigning variables for our blast project.
2. Download 10 genes from humans (our queries), and the entire sequence for the chromosome 21 genes (subject) from ncbi using the command `curl`.
3. We ran `makeblastdb` to get all the formats that `blast` needs to create a reference database, using the **Chromosome 21 fasta file**
4. Ran a blast search of one single gene (query), against our reference database (subject).

Goals for today:

1. I am going to explain **one of many ways** to run our script on a `loop`.
2. You will not create the script, you will only modify the variables of my script to adjust the commands to your own `blast_script.sh`.
3. Optimize how we download biological data from ncbi, we will say bye to `curl`, and we will use `datasets`, a function from the ncbi bioinformatic tool.

:rotating_light: **Remember** this markdown document is for you to answer specific questions and submit it as evidence of class participation. The script is a different file with an `sh` extension. This is the executable file that will run the entire pipeline. Your script should follow the best practices we covered in class.

Table of Contents

BLAST 10 genes with a loop	3
NCBI databases	7
Orthologs: one gene in five species	8
To get credit:	12

BLAST 10 genes with a loop

You already know the `while read` loop from the Chromosome 21 exercise. As a reminder:

[!TIP] Syntax: `while read` (loop over a file, line by line)

```
while read -r var1 var2
do
  COMMANDS_USING_${var1}_AND_${var2}
done < INPUT_FILE
```

:bulb: Revisit your `bash_loops_conditionals_chr21.md` document and the `while read` syntax above if needed.

We have our `10_genes.txt` file, it has two columns, where column 1 has the gene name, and column 2 the accession number:

```
LSS NM_001145436.2
SPATC1L NM_001142854.2
FTCD NM_001320412.2
GART NM_000819.5
DNAJC28 NM_017833.5
TMEM50B NM_006134.7
IFNGR2 NM_005534.4
C21orf62 NM_001162495.3
PAXBP1 NM_013329.4
BTG3 NM_006806.5
```

And your fasta files that you already downloaded with the `chr21.sh` script

```
LSS.fasta
SPATC1L.fasta
FTCD.fasta
GART.fasta
DNAJC28.fasta
TMEM50B.fasta
IFNGR2.fasta
C21orf62.fasta
```

PAXBP1.fasta
BTG3.fasta

This is one way that we can run a loop in a single script.

```
while read -r GENE ID
do
  blastn -db $WORKDIR/$INPUTDIR/chr21_database -query $WORKDIR/$INPUTDIR/${GENE}.fasta -out $WORKDIR/$OUTPUTDIR/
${GENE}_blast_output.tsv -outfmt 6
done < 10_genes.txt
```

[!WARNING] :warning: this command will substitute your single command line, where you just call one single \$INPUT file:

```
blastn -db chr21_database -query $INPUT -out blast_output.tsv
```

[!NOTE] :bulb: Notice that I wrote the GENE variable inside {}. This lets me attach text before or after the variable, so I can use it as a prefix or suffix.

In this loop I use \$GENE twice:

1. For my query, i.e. the input file (e.g. LSS.fasta, SPATC1L.fasta, FTCD.fasta...)
2. For my output filename, so I get one output file per gene (e.g. LSS_blast_output.tsv, SPATC1L_blast_output.tsv, FTCD_blast_output.tsv)

If I type ` \$GENE\_blast\_output.tsv`, the computer can't tell where the variable name ends and the text begins. It reads ` \$GENE\_blast\_output.tsv` as one variable, which doesn't exist.

The ``{}`` tell the computer that only ``GENE`` is the variable and the rest is text. ``${GENE}_blast_output.tsv`` becomes ``LSS_blast_output.tsv``: the value of ``$GENE`` plus the text ``_blast_output.tsv``.

:white_check_mark: **Best practice:** always use ``${}`` when a variable is part of a filename.

```
Reference: Zheng Zhang, Scott Schwartz, Lukas Wagner, and Webb
Miller (2000), "A greedy algorithm for aligning DNA sequences", J
Comput Biol 2000; 7(1-2):203-14.

Database: input_data/chr21.fa
      1 sequences; 46,709,983 total letters

Query= NM_006806.5 Homo sapiens BTG anti-proliferation factor 3 (BTG3),
transcript variant 2, mRNA

Length=1410

      Score      E
Sequences producing significant alignments:      (Bits) Value

chr21                                1262  0.0

>chr21
Length=46709983

Score = 1262 bits (683), Expect = 0.0
Identities = 683/683 (100%), Gaps = 0/683 (0%)
Strand=Plus/Minus

Query 728      CAGATTTTCAGAACTTATATTTCACCTCTTCCAATGTGGCACCCCTTGCCCAGAAAAAAG 787
      |||||||||||||||||||||||||||||||||||||||
Sbjct 17594335 CAGATTTTCAGAACTTATATTTCACCTCTTCCAATGTGGCACCCCTTGCCCAGAAAAAAG 17594276
```

That file has the alignment of your query (your fasta file), against the reference or subject (Chr21.fa database that `makeblastdb` created). It is the same output that the NCBI blast website gives. But we can have a better format, a `blastp` table. For this we use the flag `-outfmt` (type `blastn -h` on your command line or look at the `blastn` manual on the website to see all the flags (options) that `blastn` can do).

Modify your `blastn` command and add the `-outfmt` as follows:

```
while read -r GENE ID
do
  blastn -db $WORKDIR/$INPUTDIR/chr21_database -query $WORKDIR/$INPUTDIR/${GENE}.fasta -out $WORKDIR/$OUTPUTDIR/
${GENE}_blast_output.tsv -outfmt 6
done < 10_genes.txt
```

The output now looks like this:

NM_005534.4	chr21	100.000	691	0	0	1009	1699	33436826	33437516	0.0	1277
NM_005534.4	chr21	100.000	206	0	0	338	543	33421480	33421685	1.10e-104	381
NM_005534.4	chr21	100.000	204	0	0	1	204	33403413	33403616	1.42e-103	377
NM_005534.4	chr21	100.000	165	0	0	688	852	33432172	33432336	6.80e-82	305

Now you have 6 columns table for each gene. [Here](#) you can find the meaning of each column and what is the format 6 in blastn.

Each software has multiple options and functions. **Before running any pipeline, please make sure you read the basic manual of each program you will run.**

[!Note]Go ahead and paste the loop command with the -outfmt flag to your script

:warning: Remember to adjust the PATH variables to YOUR SCRIPT (It might not be the same as mine).

[!NOTE]To do at home: After class, run the complete blast script with the loop command, if it fails try to debug it, if after fixing typos or paths is still not working, send me an email to try to debug Once your script runs, answer the following question:

:question: Go and check if all the blast tables were generated for your 10 genes. Open one and explain what each column represent and write your interpretation of the information that the first match (row) is telling you.

Type your answer:

gene name = NM_006806.5

chromosome = chr21

match % = 100.000

142

0

0

383

524

17604999

17604858

3.44e-69

263

There are multiple ways to run a loop, the best way to get experience is practice, develop your own logic and try it!

NCBI databases

:point_right: Keep working on your `blast` script directory

:point_right: Check that your `10_genes.txt` file and your `input_data` and `analysis` directories are there.

Now that our script works, we can start to make it fancier.

🔗 *Question: Look at the `curl` lines in your script. In your own words, what is inconvenient about using this approach to download biological databases?*

Type your answer:

we have to know its download a large local file from the server (time consuming), and unzip / potentially convert it to use it (unnecessary operation), and this might just be for the sake of a few genes in it or so

`curl` downloads whatever is at a URL, so we have to build the URLs by hand, they depend on each website's structure, and we download **one gene per request**. NCBI created a command-line tool specifically for downloading biological data: `datasets`.

	curl + URL	datasets
How you ask	You build the URL by hand	You ask by gene name , accession or taxon
Many genes	One request per gene (loop)	All genes in one command
What you get	One FASTA file	A .zip "data package": sequences + a metadata report
Source	UCSC + NCBI (mixed)	NCBI only

Orthologs: one gene in five species

So far we worked only with **human** genes. But as biologists we often want to know: does this gene exist in other species? How similar is it? Genes in different species that come from the same gene in their common ancestor are called **orthologs**. They are the starting point for comparative genomics, molecular evolution and phylogenetics.

Today you will get the orthologs of the 10 genes from human chromosome 21 in four other mammals:

Common name	Scientific name
Human	<i>Homo sapiens</i>
Chimpanzee	<i>Pan troglodytes</i>
Mouse	<i>Mus musculus</i>
Pig	<i>Sus scrofa</i>
Dog	<i>Canis lupus familiaris</i>

This part of the exercise was modified from an [NCBI tutorial](#), but with our own genes. Please open that link so you can have a better picture of what we are going to do.

First, we will do it **by hand for one gene** (LSS), and once we understand every step, we put it inside a loop for the 10 genes.

[!TIP] Syntax: `datasets summary` with orthologs

```
datasets gene symbol GENE_NAME --ortholog 'SPECIES1,SPECIES2,...'
```

- `--ortholog` adds the orthologs of your gene in the species you list (scientific names, separated by commas, inside quotes), or you can use `--ortholog all` to get the complete ortholog set available for your gene.

Let's start with LSS gene:

1. Open a new file in vscode to create our `datasets.sh` script (make sure your VScode is connected to tule)
2. We will be working with **the same directory for the blast project**

Code:

```
#REMEMBER THAT YOU NEED TO PROPERLY DOCUMENT YOUR SCRIPT, NAME, VERSION, USAGE, STEPS, REQUIREMENTS ETC.
```

```
#Set up your directories VARIABLES (COPY AND PASTE THE SAME VARIABLES FROM THE BLAST SCRIPT):
```

```
WORKDIR=blast_project #Or whatever you named your $WORKDIR variable for the previous exercise
```

```
INPUTDIR=
```

```
OUTDIR=
```

```
ANALYSIS=
```

```
##NEW VARIABLE!!!
```

```
DATADIR=datasets #THIS IS A NEW DIRECTORY THAT YOU WILL CREATE FOR TODAY
```

```
#Data variables
```

```
GENE=LSS #Because we are testing this in one single gene first.
```

```
#Start your actual script:
```

```
mkdir -p $WORKDIR/$DATADIR/orthologs
```

```
#1. Download LSS data from ncbi using datasets instead of curl
```

```
datasets download gene symbol $GENE --ortholog 'homo sapiens,pan troglodytes,mus musculus,sus scrofa,canis lupus familiaris' --
```

```
filename $WORKDIR/$DATADIR/orthologs/${GENE}_orthologs.zip
```

```
unzip $WORKDIR/$DATADIR/orthologs/${GENE}_orthologs.zip -d $WORKDIR/$DATADIR/orthologs/${GENE}_orthologs
```

:warning: Notice that now we are using `unzip` instead of `gunzip`, always look at the file extensions to choose if you should use `gunzip` or `unzip`: `gunzip` is used to decompress .gz (gzip) files, while `unzip` is used to extract .zip archives.

[!TIP] Syntax: `unzip`

```
unzip FILENAME.zip -d OUTPUT_DIRECTORY
```

- Remember from the Chromosome 21 exercise: `.gz` → `gunzip`, `.zip` → `unzip`. `datasets` gives you `.zip` files.
- `-d` extracts everything into the directory you choose.

Go to the new directory that was created in your terminal.

:question: Where did the new LSS_orthologs directory was created (give the complete path that you got with `pwd`)

Type your answer:

:question: How many files, and how many directories were unzipped? List all the content inside the LSS_orthologs directory (files, directories, and files inside the directories)

Type your answer:

You will see a file named `md5sum.txt`. This is a new format for you. This a MD5 checksum file that is used to verify file integrity. Basically, it will let us know if all the files were downloaded complete and without errors. You will find this type of files in many softwares or in many protocols.

To check if your download worked type:

```
md5sum -c md5sum.txt
```

:warning: Make sure you are running this inside the directory where `md5sum.txt` is.

Type the output from `md5sum`

:question: Explore the `ncbi_dataset/data` directory, and briefly explain what information each file has:

Type your answer here

:warning: Note that the files are named `protein.faa` and `rna.fna`. **All ncbi datasets are going to give you those filenames and the same directory structures by default.** We need to rename the files adding the name of the gene, for this you can use `mv` as follows:

```
mv protein.faa LSS_protein.faa
```

This will rename your `protein.faa` file to `LSS_protein.faa`

:question: Now, following this logic, how would you add this line to your `datasets.sh` script so that every time `datasets` downloads and unzips a data package, the **protein and RNA** files are automatically renamed using the `$GENE` variable?

First explain the logic to follow (no coding at all):

Step 1:

Step 2....

Now type your code using the corresponding variables

Now that you have all your script ready and functional for LSS gene, use the same logic from the first section of this document, to create a loop for the `datasets.sh` script. The goal is to download all the orthologs proteins and rna datasets from each gene, in a single run, including renaming the files. **SAVE THIS FILE AS `datasets_loop.sh`.**

:crossed_fingers: Let's run it!!!

If it failed, write here the main issues that you found and how did you fix them

To get credit:

Submit the PDF version of this markdown with your answers to canvas, under `datasets` assignment.